

Réseaux de Neurones Récurrents (RNN)

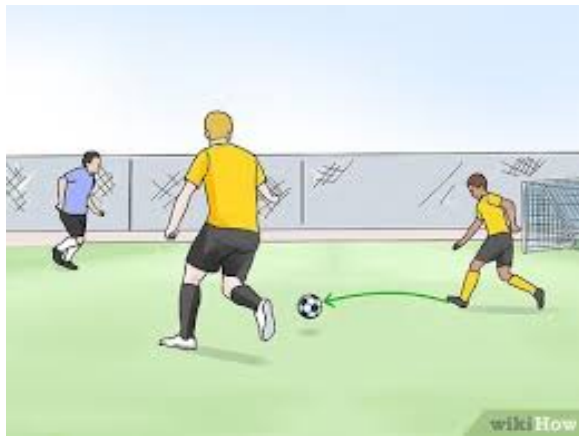
Traitement des séquences

Plan

- ① Introduction
- ② Neurones et couches récurrents
- ③ Cellules mémoire
- ④ Séquences d'entrée et de sortie
- ⑤ Entraînement des RNN
- ⑥ RNN profonds
- ⑦ Traitement des longues séquences
- ⑧ Cellules LSTM et GRU
- ⑨ Résumé

Introduction

Prédire ce qui vient ensuite est quelque chose que nous faisons en permanence.



Un joueur frappe le ballon ; l'autre réagit et **anticipe**.

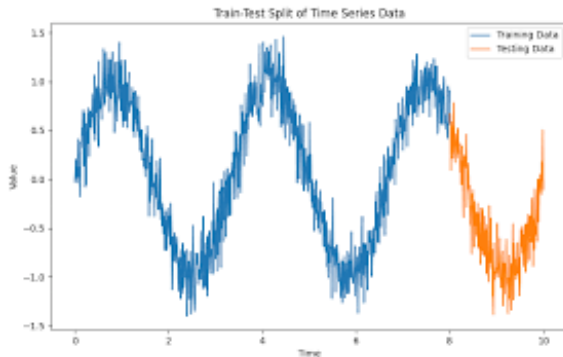
Prédire ce qui vient ensuite est quelque chose que nous faisons en permanence.



On finit la phrase d'un ami, on devine le mot suivant en tapant.

Réseaux de Neurones Récurrents (RNN)

Les RNN sont utilisés dès que les données ont une structure **temporelle** ou **séquentielle**.



Séries temporelles : cours boursiers, météo, capteurs (prévision, anomalies).

Les RNN sont utilisés dès que les données ont une structure **temporelle ou séquentielle**.



Systèmes autonomes : conduite, robotique (séquences de capteurs).

Applications

Les RNN sont utilisés dès que les données ont une structure **temporelle ou séquentielle**.



Langage naturel : traduction, génération de texte, sentiment, parole.

Applications

Les RNN sont utilisés dès que les données ont une structure **temporelle ou séquentielle**.



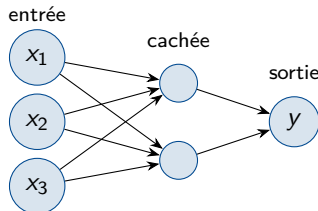
Audio & vidéo : transcription, sous-titrage, reconnaissance d'activité.

Limites des réseaux à propagation directe

Réseaux à propagation directe (feedforward)

Les activations circulent dans **une seule direction** : entrée $\rightarrow$ sortie.

- Pas de notion de « précédent » ou « suivant » ; chaque entrée est traitée isolément.
- Inadaptés aux séquences : le **passé** devrait influencer le présent et le futur.



RNN

Introduisent des **connexions récurrentes** pour que l'information persiste d'un pas de temps à l'autre.

Neurones et couches récurrents

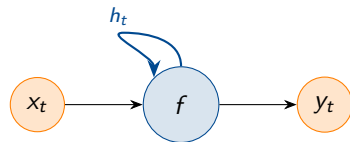
Neurones et couches récurrents

Un RNN ressemble à un réseau feedforward, mais avec des **connexions qui bouclent**.

Un seul neurone récurrent à chaque pas t

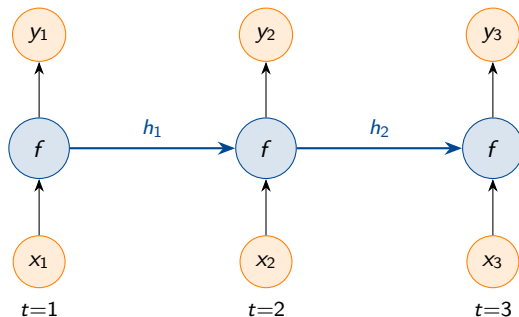
- 1 Reçoit l'entrée x_t et l'état caché précédent h_{t-1} .
- 2 Produit la sortie y_t et l'état caché h_t .
- 3 h_t est renvoyé au pas suivant. h_0 est généralement 0.

Vue **enroulée** : une cellule, une boucle.



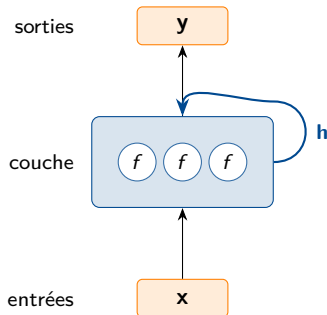
Déroulement dans le temps

- On dessine le même neurone **une fois par pas de temps** le long d'un axe temporel.
- On appelle cela **dérouler le réseau dans le temps**.
- Transforme la boucle récurrente en une **chaîne** ; permet l'entraînement par **rétropropagation**.



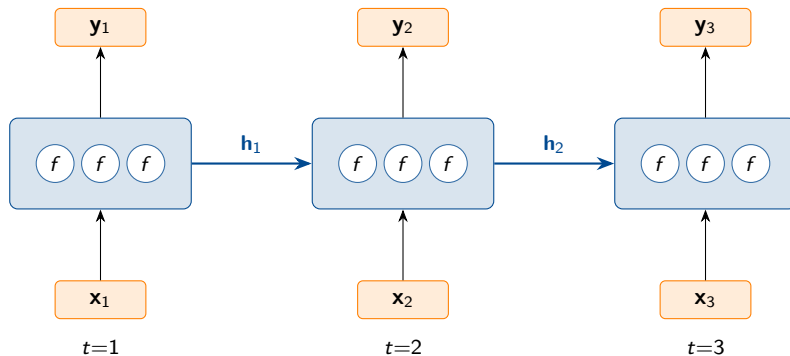
Une couche de neurones récurrents (enroulée)

On peut regrouper plusieurs neurones récurrents en une **couche**.



La couche reçoit un vecteur d'entrée **x**, produit un vecteur de sortie **y**, et renvoie son état caché **h** à elle-même.

Une couche de neurones récurrents (déroulée)



La couche entière est copiée à chaque pas de temps. Le vecteur d'état caché h_t est transmis d'un pas au suivant.

Feedforward : un seul jeu de poids W (et biais b) sur l'entrée. **Couche récurrente** : deux jeux de poids :

- W_x : appliqué à l'entrée courante x_t ,
- W_h : appliqué à l'état caché précédent h_{t-1} .

Équation de récurrence

$$h_t = f(W_x^\top x_t + W_h^\top h_{t-1} + b)$$

f = activation (ex. tanh). Les mêmes W_x, W_h, b sont **partagés à tous les pas de temps**.

Cellules mémoire

Une **cellule mémoire** est la partie du réseau qui **conserve un état à travers les pas de temps**. À chaque pas, elle :

- Reçoit l'entrée courante et (éventuellement) l'état caché précédent,
- Met à jour son état interne,
- Produit une sortie (qui peut différer de l'état interne complet).

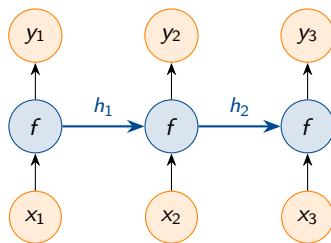
L'**état caché** (transmis au pas suivant) et la **sortie** (pour la prédiction) peuvent différer. Dans un RNN simple, ils sont identiques ; dans un LSTM/GRU, la cellule contrôle ce qu'elle retient et ce qu'elle produit en sortie.

Différentes architectures

Séquence à séquence (Seq2Seq)

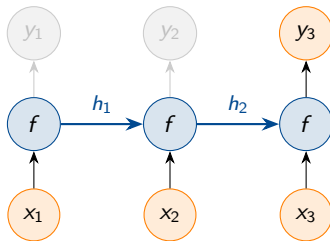
Entrée : séquence. **Sortie** : séquence (une par pas). **Exemples** :

- Prédiction de séries temporelles : les N derniers prix $\rightarrow$ jour(s) suivant(s).
- Modélisation du langage : tokens $\rightarrow$ token suivant à chaque pas.



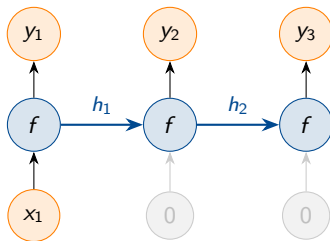
Séquence vers vecteur : une séquence $\rightarrow$ un vecteur (ex. dernière sortie uniquement).

- Sentiment (mots $\rightarrow$ label), classification vidéo (images $\rightarrow$ classe), plongement de documents.



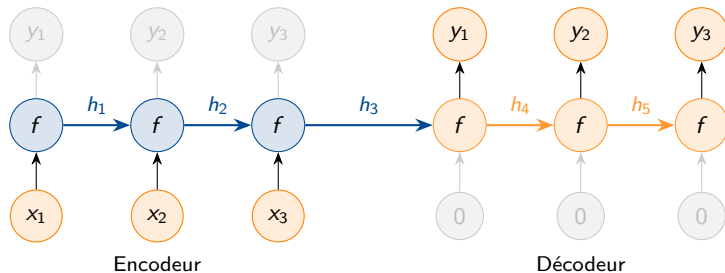
Vecteur vers séquence : un vecteur $\rightarrow$ une séquence.

- Légendage d'images (image $\rightarrow$ mots), génération de mélodie (graine $\rightarrow$ notes).



Encodeur-Décodeur ($\text{Seq} \rightarrow \text{Vec} \rightarrow \text{Seq}$)

- **Encodeur** : séquence d'entrée $\rightarrow$ un vecteur de taille fixe.
- **Décodeur** : ce vecteur $\rightarrow$ séquence de sortie.
- Utilisé en traduction automatique, résumé, et de nombreuses tâches seq2seq.

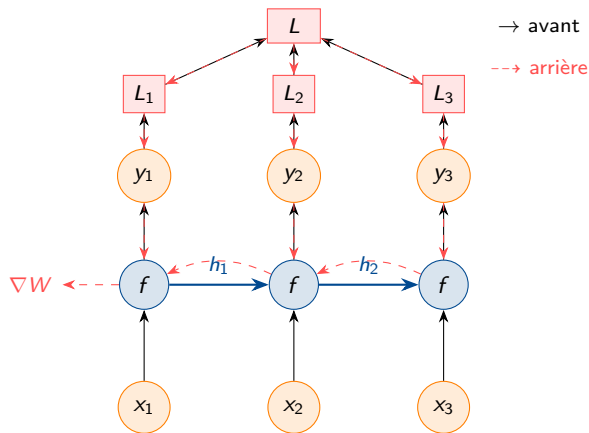


Rétropropagation à travers le temps

Rétropropagation à travers le temps (BPTT)

- 1 **Passe avant** : calcul des sorties et des états cachés.
- 2 **Perte** : $L(y_1, \dots, y_T)$.
- 3 **Passe arrière** : les gradients remontent à travers le réseau déroulé.
- 4 **Mise à jour** : actualisation de W_x, W_h, b .

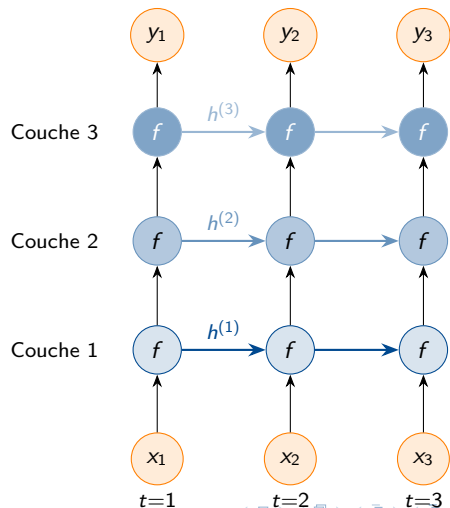
Mêmes paramètres à chaque pas $\Rightarrow$ les gradients se **somment** sur les pas de temps.



RNN profonds

RNN profonds

- **Empiler** plusieurs couches récurrentes : la sortie d'une couche $\rightarrow$ l'entrée de la suivante.
- Plus profond $\Rightarrow$ motifs plus complexes.
- Plus de paramètres, risque accru de gradients qui explosent/s'évanouissent.
- Profondeur généralement modeste (2–4 couches).



Deux problèmes principaux

Deux problèmes principaux

Longues séquences $\Rightarrow$ déroulement sur de nombreux pas (réseau effectivement très profond).

- ❶ **Gradients instables** : entraînement lent ou instable (explosion/évanouissement).
- ❷ **Mémoire à court terme** : après de nombreux pas, l'information ancienne est « oubliée » ; le modèle peine à utiliser un contexte lointain.

1. Problème des gradients instables

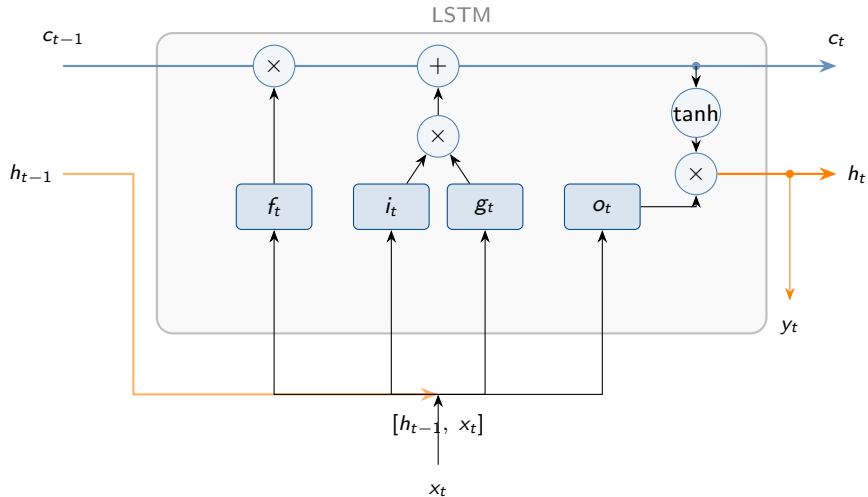
- Utile : meilleure initialisation, optimiseurs plus rapides (Adam), dropout.
- **Activations** : ReLU peut rendre les RNN *plus* instables (mêmes poids à chaque pas $\Rightarrow$ les activations peuvent exploser). Utiliser tanh (saturante).
- **Explosion du gradient** : le **gradient clipping** (par norme ou valeur) est efficace.
- **Normalisation** : la batch norm est moins naturelle pour les RNN ; la **layer normalization** est souvent préférée.

2. Problème de la mémoire à court terme

À chaque pas, l'état caché est transformé ; de l'information est perdue. Après de nombreux pas : presque **aucune trace des premières entrées**. **Solution** : des cellules avec une **mémoire à long terme** qui apprennent quoi stocker, oublier et produire en sortie. Exemples principaux : **LSTM** et **GRU**.

LSTM

LSTM : Architecture complète



LSTM : deux vecteurs d'état

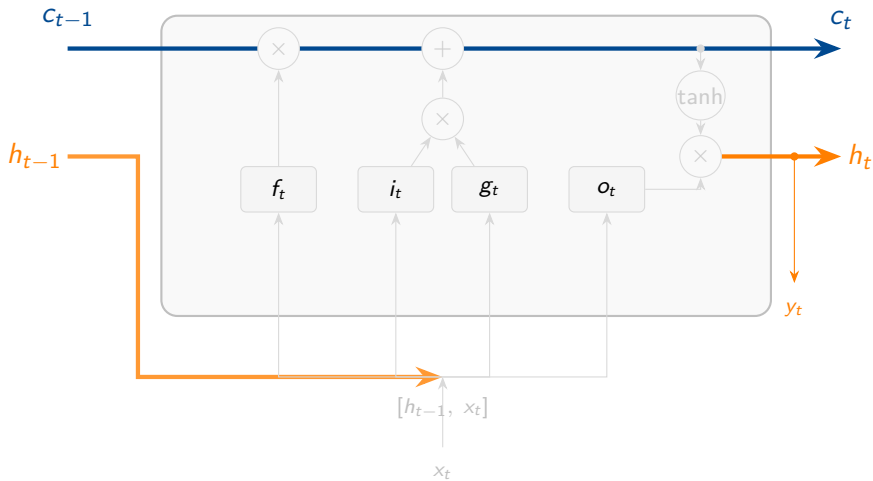
Long Short-Term Memory (LSTM) : état interne scindé en deux vecteurs :

- h_t : **état à court terme** (sortie de la cellule),
- c_t : **état à long terme** (état de la cellule).

Le réseau apprend :

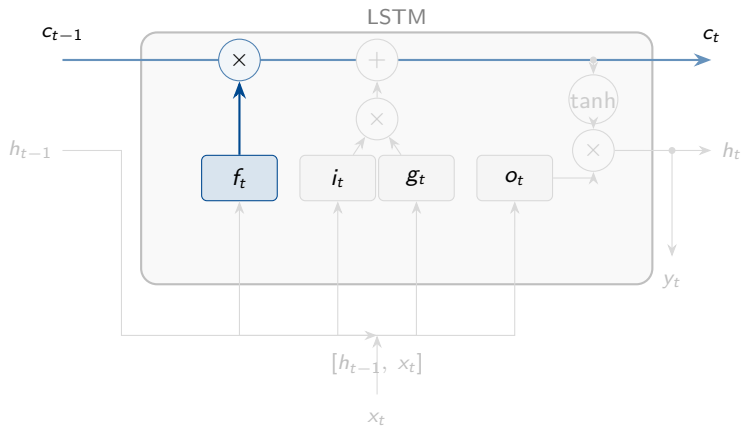
- Quoi **stocker** dans l'état à long terme,
- Quoi **oublier**,
- Quoi **lire** et exposer en sortie.

LSTM : deux vecteurs d'état



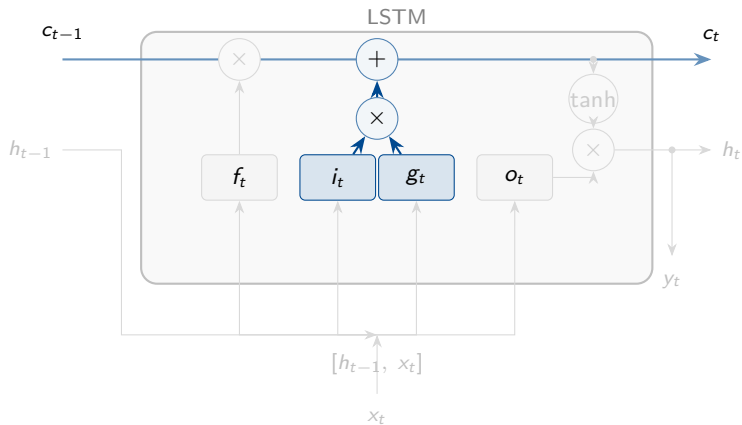
LSTM Étape 1 : Porte d'oubli

Porte d'oubli : f_t contrôle quelles parties de c_{t-1} **effacer**.



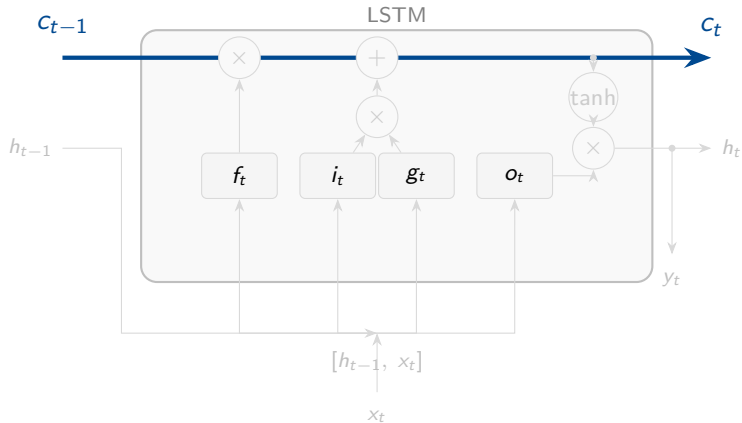
LSTM Étape 2 : Porte d'entrée + Candidat

La **porte d'entrée** i_t sélectionne quelles parties du candidat g_t **ajouter** à l'état de la cellule.



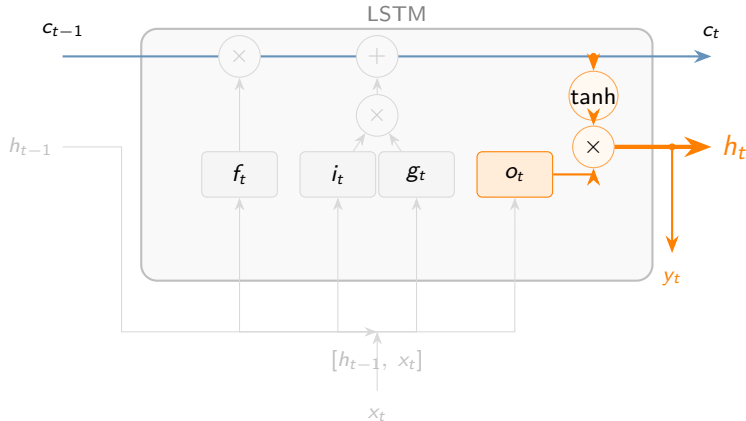
LSTM Étape 3 : L'état de cellule traverse

c_t est transmis au pas suivant **sans activation** — l'information peut voyager sur de longues distances.

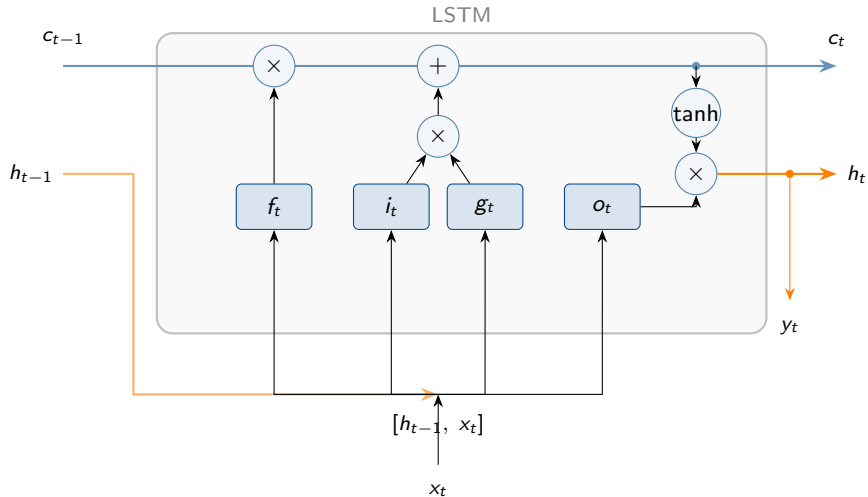


LSTM Étape 4 : Porte de sortie

La **porte de sortie** o_t contrôle ce qui est lu : $h_t = o_t \odot \tanh(c_t)$.



LSTM : Architecture complète



Équations du LSTM

Soit $\sigma = \text{sigmoïde}$, $[h_{t-1}, x_t] = \text{concaténation}$, $\odot = \text{produit élément par élément}$:

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f) \quad (\text{oubli})$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i) \quad (\text{entrée})$$

$$g_t = \tanh(W_g [h_{t-1}, x_t] + b_g) \quad (\text{candidat})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o) \quad (\text{sortie})$$

$$h_t = o_t \odot \tanh(c_t)$$

Pourquoi $\tanh$ et σ dans les LSTM ?

Dans un cours précédent, nous avons vu que $\tanh$ et σ peuvent causer l'**évanouissement du gradient** dans les réseaux feedforward profonds. Alors pourquoi les utiliser ici ?

σ (sigmoïde) pour les portes : f_t, i_t, o_t

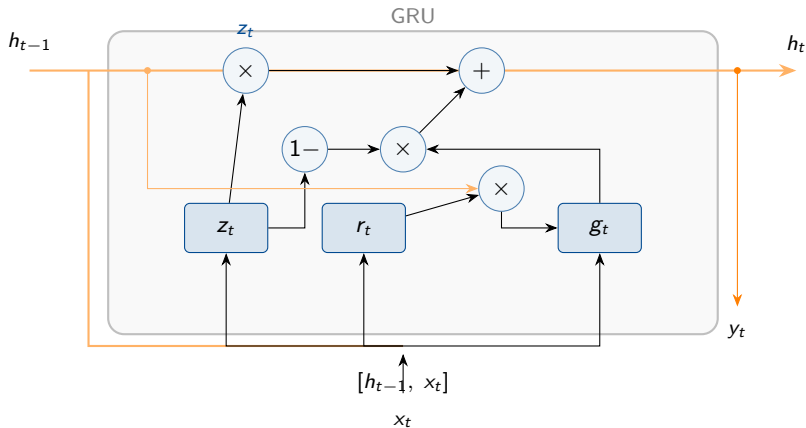
- Les portes doivent produire des valeurs dans $[0, 1]$ pour agir comme des **interrupteurs souples**.
- 0 = complètement fermé, 1 = complètement ouvert.
- Aucune autre activation standard n'a cette plage. La sigmoïde est le choix naturel.

$\tanh$ pour le candidat g_t et la lecture de la cellule

- Sortie dans $[-1, 1]$: centre les valeurs autour de 0, évite la croissance non bornée de c_t .
- Sans borne, l'état de cellule pourrait exploser à travers les pas de temps.

L'état de cellule c_t traverse des **opérations linéaires** ($\times, +$), *pas* $\tanh/\sigma$. Les gradients voyagent le long de c_t sans écrasement répété $\Rightarrow$ l'évanouissement du gradient est atténué.

GRU : Architecture complète

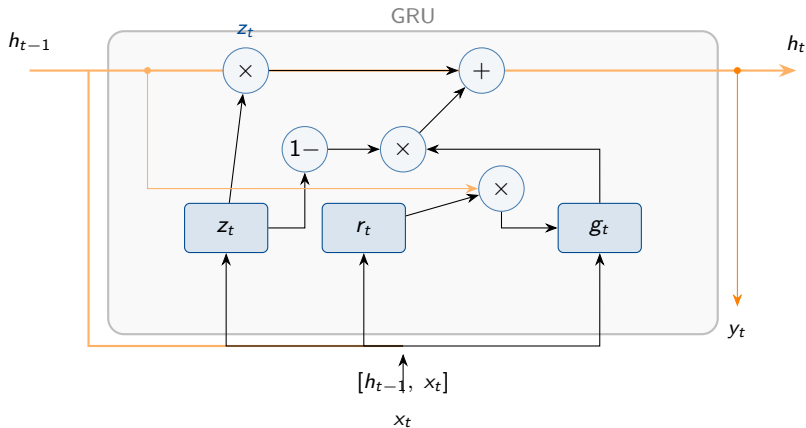


GRU

Gated Recurrent Unit (GRU) :

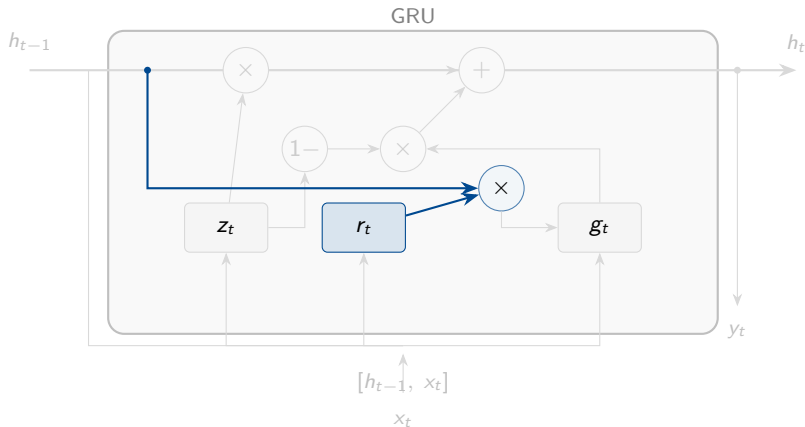
- **Un seul état** h_t (pas de c_t séparé).
- **Une seule porte** z_t (mise à jour) : quand $z_t \approx 1$ on garde l'état précédent ; quand $z_t \approx 0$ on prend le nouveau candidat (« vider la case » avant de stocker).
- **Pas de porte de sortie** : h_t complet est la sortie. La **porte de réinitialisation** r_t contrôle quelle partie de h_{t-1} la couche candidat voit.

GRU : Architecture complète



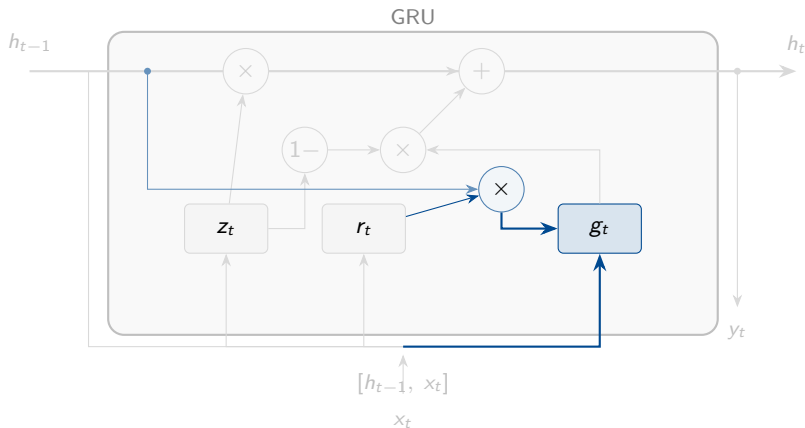
GRU Étape 1 : Porte de réinitialisation

La **porte de réinitialisation** r_t contrôle la quantité de h_{t-1} que le candidat peut voir.



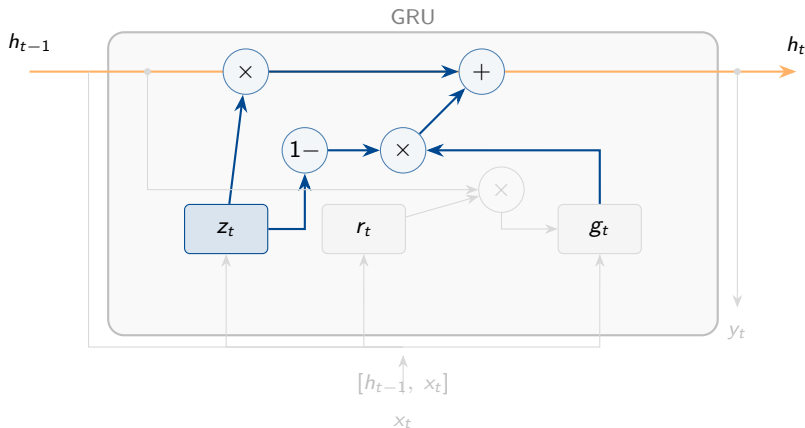
GRU Étape 2 : Candidat g_t

Le **candidat** g_t propose un nouvel état caché, en utilisant $r_t \odot h_{t-1}$ et x_t .



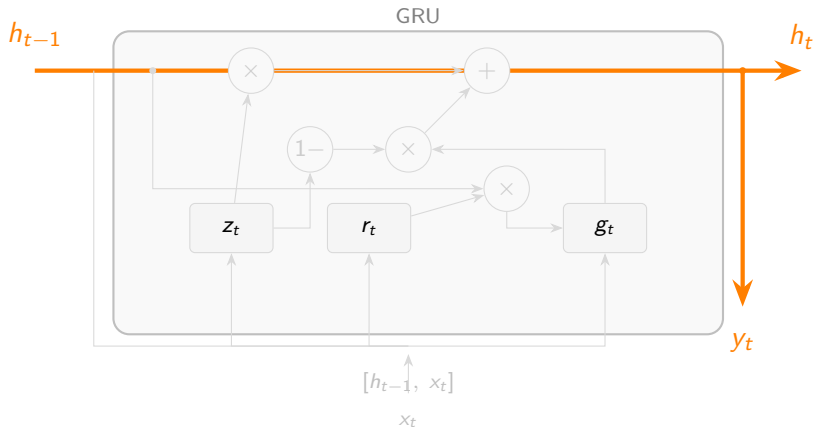
GRU Étape 3 : Porte de mise à jour

La **porte de mise à jour** z_t mélange ancien état et candidat : $h_t = z_t \odot h_{t-1} + (1-z_t) \odot g_t$.



GRU Étape 4 : Sortie

Le nouveau h_t est à la fois l'**état caché** pour le pas suivant et la **sortie** y_t .



$$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z) \quad (\text{porte de mise à jour})$$

$$r_t = \sigma(W_r [h_{t-1}, x_t] + b_r) \quad (\text{porte de réinitialisation})$$

$$g_t = \tanh(W [r_t \odot h_{t-1}, x_t] + b) \quad (\text{candidat})$$

$$h_t = (1 - z_t) \odot g_t + z_t \odot h_{t-1}$$

h_t est un mélange de l'état précédent et du candidat, contrôlé par z_t .

- **RNN** : connexions récurrentes pour que la même unité traite une séquence et transmette l'information à travers le temps.
- **Déroutement + BPTT** : entraînement par rétropropagation à travers la chaîne déroulée.
- **Cellules mémoire** : LSTM/GRU contrôlent ce qu'il faut retenir et produire en sortie.
- **Architectures** : seq2seq, seq2vec, vec2seq, encodeur-décodeur.
- **Longues séquences** : utiliser tanh, gradient clipping, layer norm, et LSTM/GRU pour un entraînement stable et des dépendances à long terme.

Merci / Questions ?